



Programa Certified AI-LLM Solution Architect

Curso Diseño de Infraestructura Escalable

Claves para construir plataformas eficientes y adaptables en la nube

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



CAPÍTULO 2

SESIÓN 3

3 HORAS

Kubernetes, Docker y LLMs en la Nube

Sistemas Distribuidos y Contenedores para Infraestructura Escalable de IA



Docker



K8s



Azure



GCP



AWS

Agenda — Sesión 3

Tres bloques de trabajo • 3 horas de alto impacto

01

60 min

Introducción a Kubernetes y Docker

- ▶ ¿Por qué contenedores?
- ▶ Conceptos: Image, Container, Pod, Node
- ▶ Configuración de clústeres para LLMs
- ▶ Ollama: LLM local sin costo

02

80 min

Práctica con Contenedores

- ▶ Docker local → 3 nubes (mismo Dockerfile)
- ▶ Kubernetes local con Minikube
- ▶ AKS · GKE · EKS — deploy idéntico
- ▶ FastAPI como LLM Gateway

03

40 min

Costos, ROI y Riesgos

- ▶ 7 criterios de evaluación cloud
- ▶ Análisis de costos 100K req/mes
- ▶ ROI 229% en 18 meses
- ▶ Gestión de riesgos técnicos



Break 15 min entre Bloque 1 y 2

¿Por qué Contenedores para LLMs?

Tema 1 · El problema que resuelven Docker y Kubernetes



El Problema

- ✗ "En mi máquina funciona" — conflictos de dependencias
- ✗ CUDA 11 vs CUDA 12 — incompatibilidades de GPU
- ✗ Llama 3 necesita 16GB RAM — ¿cómo escalar?
- ✗ Deploy manual en 3 equipos diferentes
- ✗ Sin aislamiento → un modelo afecta a otro



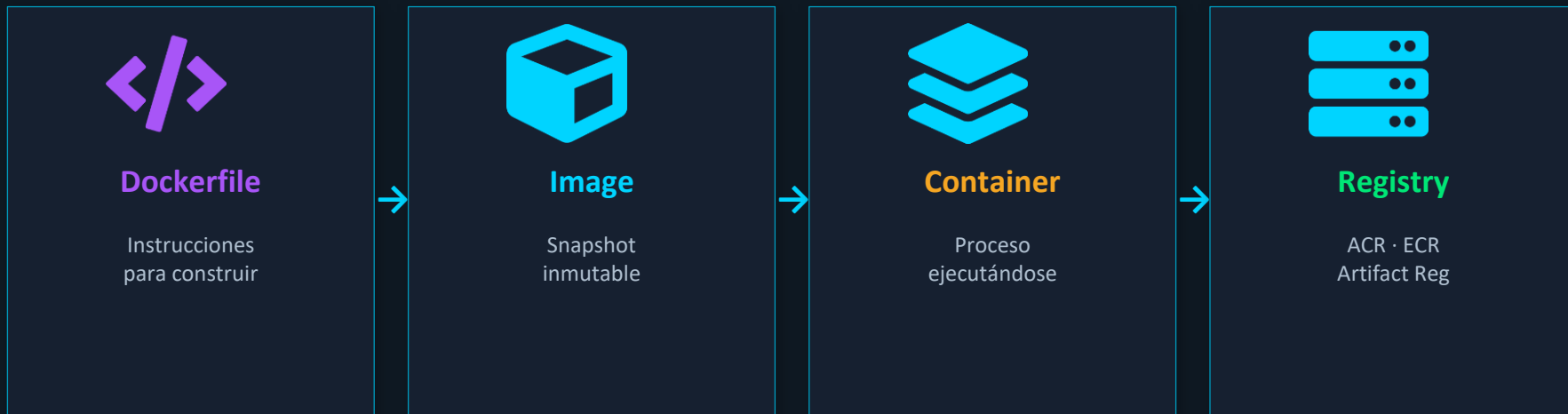
La Solución

- ✓ Imagen Docker = entorno reproducible 100%
- ✓ GPU drivers gestionados por el host, no el contenedor
- ✓ HPA de K8s: 2 → 10 pods en segundos automático
- ✓ 1 YAML → deploy en Azure, GCP y AWS idéntico
- ✓ Namespaces → aislamiento garantizado por diseño



Docker: Conceptos Fundamentales

Tema 1 · Del Dockerfile a un LLM corriendo en producción



Dockerfile — LLM Gateway

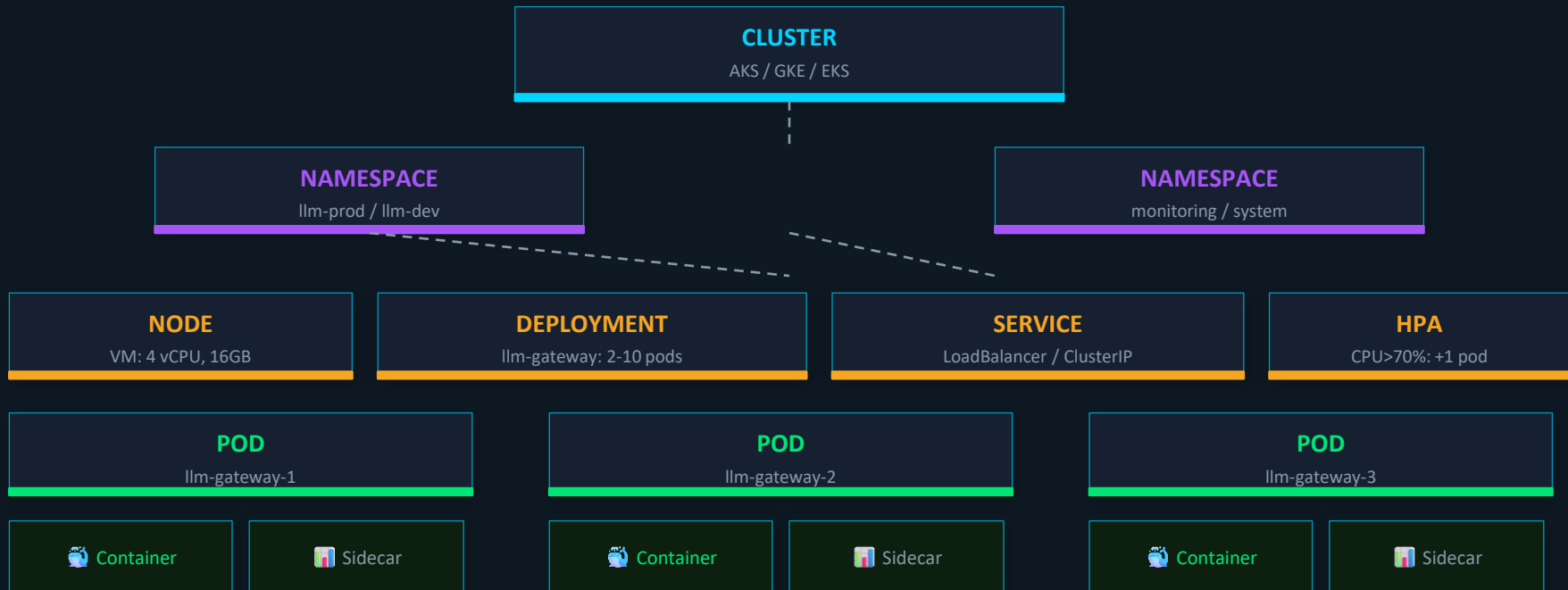
```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

Comandos esenciales

```
docker build -t llm-gateway .           # Construir imagen
docker run -p 8000:8000 llm-gateway     # Ejecutar contenedor
docker push myregistry/llm-gateway      # Subir al registry
docker-compose up -d                    # Stack completo local
```

Kubernetes: Orquestación a Escala

Tema 1 · De un Pod a un clúster multi-nube con auto-escalado



Ollama + FastAPI: LLM Local sin Costo

Tema 1 · Práctica inmediata — cero credenciales, cero costo, máximo aprendizaje



Ollama — Setup en 3 pasos

```
# Instalar Ollama

curl -fsSL https://ollama.ai/install.sh | sh

# Descargar modelo (2GB)

ollama pull llama3.2:3b

# Ejecutar y consultar

ollama serve

curl http://localhost:11434/api/chat \

  -d '{"model": "llama3.2:3b", ...}'
```

Modelos recomendados para clase:

llama3.2:3b 2GB ★ Recomendado aula

mistral:7b 4GB Buenas instrucciones

phi3:mini 2.3GB Rápido y eficiente



FastAPI Gateway — Endpoints

GET

`/health`

Estado del servicio

GET

`/models`

Modelos disponibles

POST

`/chat`

Chat con LLM

POST

`/chat/stream`

Streaming SSE

POST

`/embeddings`

Vectores semánticos

```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```

Mismo Dockerfile → Tres Nubes

Tema 2 · Portabilidad como principio: build once, deploy anywhere



💡 Mismo Dockerfile para las 3 nubes — solo cambian las credenciales y el registry de destino

Kubernetes Gestionado: AKS vs GKE vs EKS

Tema 2 · El mismo YAML de Kubernetes funciona en las 3 plataformas

Criterio	AKS (Azure)	GKE (Google)	EKS (AWS)
Versión K8s	1.29+	1.29+ (Autopilot)	1.29+
Control Plane	Gratuito	~\$0.10/hora	\$0.10/hora
Nodos	B2s: \$35/mes	e2-medium: \$27/mes	t3.medium: \$33/mes
Auto-scaling	HPA + KEDA	HPA + VPA	HPA + Karpenter
GPU support	NC series NVIDIA	A100/T4 nativo	p3/g4 instances
Free tier	\$200 crédito	\$300 crédito	12 meses gratis
Setup estudiante	az aks create	gcloud container	eksctl create



Mismo deployment.yaml para las 3 plataformas — kubectl apply funciona igual en AKS, GKE y EKS

Deployment YAML — El Mismo para las 3 Nubes

Tema 2 · Un solo archivo de configuración, infinitas posibilidades de despliegue

kubernetes/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: llm-gateway
  namespace: llm-prod
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llm-gateway
  template:
    spec:
      containers:
        - name: llm-gateway
          image: REGISTRY/llm-gateway:v1
          ports:
            - containerPort: 8000
```

HPA — Auto-Escalado

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          averageUtilization: 70
```

kubectl — Comandos esenciales

<code>kubectl apply -f kubernetes/</code>	← Desplegar todo
<code>kubectl get pods -n llm-prod</code>	← Ver pods
<code>kubectl logs -f deploy/llm-gateway</code>	← Ver logs
<code>kubectl scale deploy llm-gateway --replicas=5</code>	← Escalar
<code>kubectl get svc -n llm-prod</code>	← Ver IP externa
<code>kubectl top pods</code>	← Uso de recursos



7 Criterios para Evaluar tu Plataforma Cloud

Tema 3 · Framework de decisión para arquitectos de LLM en LATAM



01

25%

Costo Total (TCO)

Cómputo + storage + red + API calls + soporte. Proyección a 12 y 36 meses.



02

20%

Rendimiento & Latencia

P95/P99 latency, throughput tokens/seg, GPU availability, cold start time.



03

15%

Escalabilidad

HPA, VPA, KEDA. Burst capacity. Multi-región. Auto-scaling sin intervención.



04

15%

Seguridad & Cumplimiento

SOC2, ISO27001, GDPR/LGPD, encriptación at-rest y in-transit, VPC privada.



05

10%

Integración Ecosistema

LangChain, APIs nativas, monitoreo (Grafana/LangSmith), CI/CD, IaC.



06

10%

Soporte & SLA

Uptime SLA (99.9% vs 99.99%), soporte técnico LATAM, documentación en ES.



07

5%

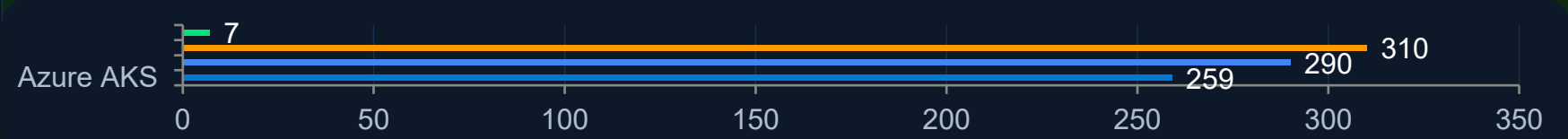
Vendor Lock-in Risk

Portabilidad del código, estándares abiertos (OCI, K8s), exit strategy.

Análisis de Costos: 100K Requests/Mes

Tema 3 · Escenario real: LLM Gateway para empresa LATAM mediana

Componente	Azure (AKS)	GCP (GKE)	AWS (EKS)	Ollama Local
Cómputo (nodos)	\$120	\$98	\$105	\$0
Control Plane K8s	\$0	\$73	\$73	\$0
API LLM (tokens)	\$85	\$72	\$80	\$0
Storage + Registry	\$18	\$15	\$17	\$5
Red (egress)	\$20	\$18	\$20	\$2
Monitoreo	\$16	\$14	\$15	\$0
TOTAL/MES	\$259	\$290	\$310	~\$7



ROI y Gestión de Riesgos

Tema 3 · Modelo de decisión para proyectos LLM en entorno corporativo



Retorno de Inversión

229%

ROI a 18 meses

3.6m

Payback period

\$47K

Beneficio neto anual

Beneficios cuantificados (empresa 500 empleados):

Automatización soporte IT	+\$28K/año
Reducción tiempo análisis	+\$15K/año
Menor error humano	+\$8K/año
Time-to-market -40%	+\$12K/año
Infra cloud vs on-premise	\$16K/año



Riesgos y Mitigaciones

ALTO

Vendor Lock-in

Usar K8s estándar + Terraform multi-cloud

ALTO

Costo escalado

Budget alerts + HPA limits + spot nodes

MED

Latencia de red

Región LATAM (BrazilSouth/us-east-1)

ALTO

Datos sensibles

VPC privada + Presidio PII + cifrado KMS

MED

Alucinaciones LLM

RAG + guardrails + validación humana

MED

Disponibilidad API

Fallback Ollama + circuit breaker

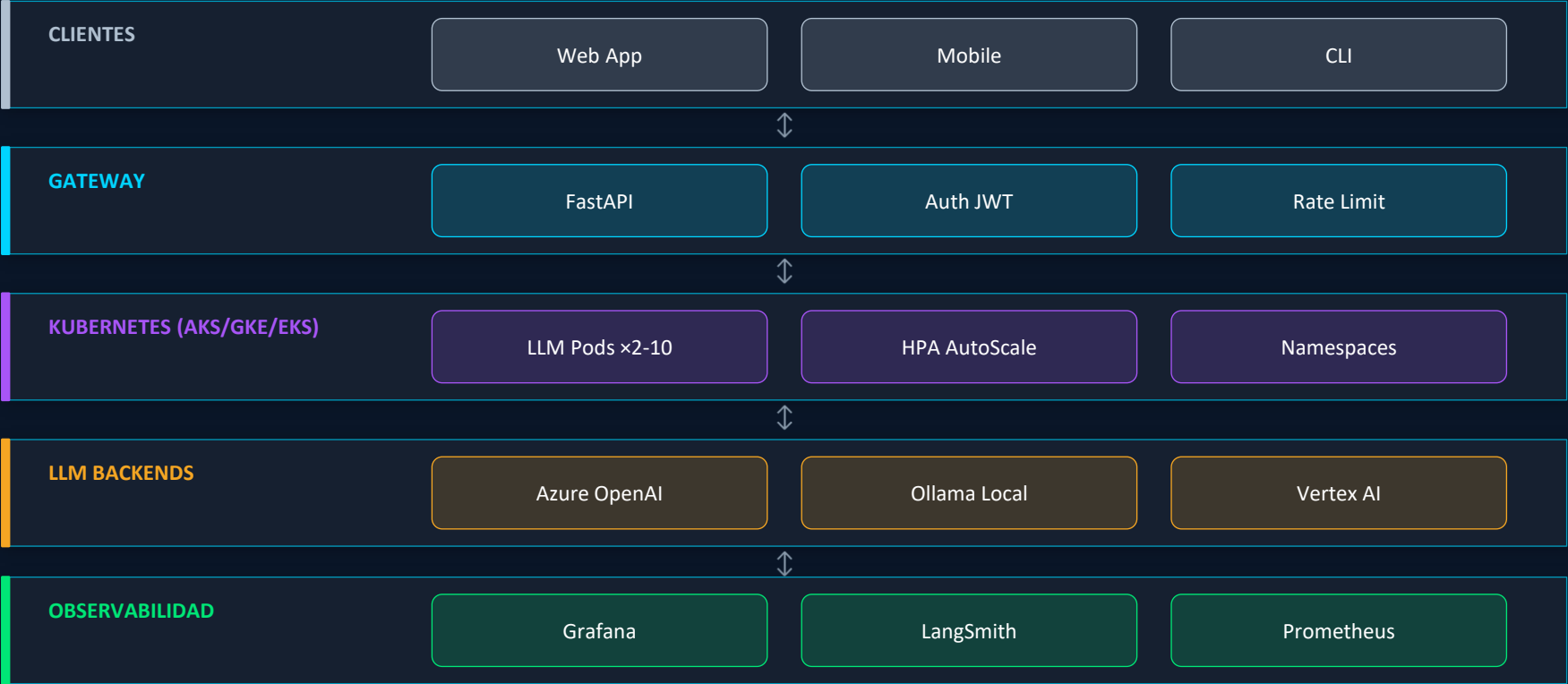
BAJO

Skill gap equipo

Training BSG + pair programming

Arquitectura Referencia: LLM en Producción

Visión integral — del código local al clúster cloud multi-zona





Resumen de la Sesión 3



Docker: Dockerfile único portable a Azure, GCP y AWS



Kubernetes: Pods, HPA, Services — mismo YAML en las 3 nubes



Ollama + FastAPI: LLM local corriendo sin costo



7 criterios de evaluación cloud con pesos diferenciados



Costos reales: Azure \$259 · GCP \$290 · AWS \$310 · Local ~\$7/mes



ROI 229% en 18 meses con payback de 3.6 meses



Recursos del Curso

Repositorio GitHub: github.com/bsg-institute/session3-infra-llm

Guía Ollama: [docs/OLLAMA_GUIDE.md](#) en el repo

Calculadora costos: [python scripts/cost-calculator.py](#)

Tests automatizados: [bash scripts/test-api.sh](#)